

ECE 364 SP26 — Birdwatching Project Report

Siqi Li(siqil10) Xin Luo(xinl12) Yijiang Yao(yao36)

Final Kaggle Score: 0.76934

Total Parameters: 9,974,830 / 10M (TinyViT-micro 5,251,203 + EfficientNet-B0 4,723,627)

1 Problem Overview

The task is 555-way fine-grained bird species classification with single-box localization on roughly 24,000 labeled training images and 24,600 unlabeled test images. For every test image our system must emit a class label, a normalized $xyxy$ bounding box in $[0, 1]^4$, and a confidence score, all under a hard 10M-parameter cap. That cap is the dominant design constraint — it immediately rules out the kind of large pretrained transformers used in most fine-grained competitions and pushes the design toward compact CNN/ViT hybrids combined with aggressive test-time strategies.

2 What We Did

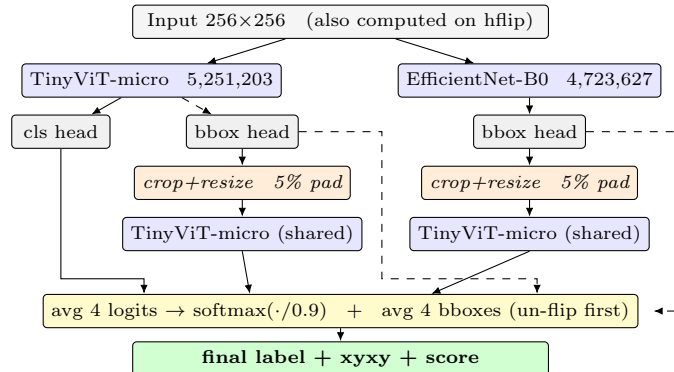
Data pipeline. We wrote a custom `BirdsDataset` that parses the CUB-style annotation files (`images.txt`, `image_class_labels.txt`, `bounding_boxes.txt`, `{train,val,test}_split.txt`). Boxes are normalized to $[0, 1]$ on load. During training we apply a *bbox-safe random crop* (scale 0.6–1.0 of image area, applied with $p=0.7$) that is constructed to always contain the ground-truth box; coordinates are then re-normalized to the cropped frame. This is followed by resize to 256×256 , random horizontal flip with bbox correction $x \rightarrow 1-x$, and color jitter (± 0.3 brightness/contrast/saturation, ± 0.1 hue). The bbox-safe crop was the single most impactful preprocessing change: it acts as a strong spatial augmentation without ever cutting off the bird.

Architecture. All five backbones we tried share the same two-head template defined in `model.py`: a backbone $\rightarrow$ global average pool $\rightarrow$ classification head (Dropout 0.3 + Linear $\rightarrow$ 555) and bbox head (Dropout 0.2 + Linear $\rightarrow$ 4 + Sigmoid). The backbones we evaluated were MobileNetV2 (3.41M), EfficientNet-B0 (4.72M), a custom **TinyViT-PSM** (~ 5.9 M) variant that captures per-block attention maps and uses attention-rollout for part selection, **TinyViT-micro** (5.25M, via `timm's tiny_vit_5m_224.dist_in22k` — our best single model), and EfficientFormerV2-S1 (~ 6.1 M).

Loss and training. The bbox head is supervised with our own **GIoU** loss (we also implemented Smooth-L1 and a $0.5 \times \text{L1} + \text{GIoU}$ combination behind a single `BBoxLoss` switch). Classification uses cross-entropy with label smoothing 0.05. Optimization uses AdamW with two parameter groups: backbone at 10^{-4} , heads at 10^{-3} , weight decay 10^{-4} , and a $2 \times$ weight on the bbox term. We use a 2-epoch linear warm-up followed by cosine annealing to 10^{-6} over 12 total epochs. An **Exponential Moving Average** of the weights (decay 0.999) is maintained in parallel and evaluated each epoch; the better of EMA and live weights is checkpointed. After the main run we finetune for 3 more epochs on $\text{train} \cup \text{val}$ at $\text{lr } 10^{-5}$ and write the EMA weights as the final checkpoint.

Inference ensemble. The submission script (`predict_final.py`) loads two checkpoints: `tinyvit_micro_final.pth` (5.25M, the only classifier) and `final_model_efficientnet_b0.pth` (4.72M, used only for its bbox head as a second localization cue). For each test image it computes four classification views — (1) full image $\rightarrow$ TinyViT, (2) crop around TinyViT's predicted box with 5% padding $\rightarrow$ TinyViT, (3) horizontally-flipped full image $\rightarrow$ TinyViT, (4) crop around EfficientNet's predicted box on the flipped image $\rightarrow$ TinyViT — averages the four logit vectors, applies softmax at temperature 0.90, and averages the four boxes (flipped boxes are un-flipped before averaging). The two-model total is 9,974,830 parameters, under cap by about 25k.

3 Final System Architecture



Solid arrows carry classification logits (one full-image view + two crop-refine views on TinyViT-micro, $\times 2$ with hflip = 4); dashed arrows carry bounding boxes that are averaged separately. **TinyViT-micro is the only classifier** — the same weights are reused for the full image and both crops. EfficientNet-B0 contributes only its bbox head as a second localization cue. Total = 9,974,830 parameters, $\sim 25k$ under the 10M cap.

4 What We Tried and Whether It Worked

Experiment	Outcome
MobileNetV2 + Smooth-L1, 5 epochs	0.197 val Top-1; first working pipeline
TinyViT-micro + GIoU loss	Val jumps to ~ 0.47 ; Kaggle 0.70378
<code>img_size</code> 224 $\rightarrow$ 256	+ ~ 0.015 val mAP; Kaggle 0.72501
4-scale feature fusion variant	0.72783 (marginal)
Bbox-safe random crop ($p=0.7$)	Large IoU gain; val mAP ≥ 0.43
CutMix (cls-only mode)	No gain on fine-grained; disabled
EMA (decay 0.999)	Consistent +0.5–1% val; kept
Label smoothing 0.05	Small but positive; kept
Finetune 3 ep on trainUval	$\sim +0.5$ –1% on Kaggle
2-model basic ensemble	0.73319
4-view crop-refine TTA	0.75717 single; 0.76934 ensemble
TinyViT-PSM attention-rollout parts	No clear gain; dropped
EfficientFormerV2-S1	Slightly below TinyViT; used only as bbox

The biggest single jump came from the **crop-refine inference pattern**: running the same classifier a second time on the region predicted by the bbox head sharpens fine-grained accuracy substantially, because the crop removes background clutter while preserving the spatially-local features (beak shape, plumage, wing bars) that distinguish closely related species. Strong augmentation, especially the bbox-safe crop, was the second-biggest contributor. EMA, label smoothing, and the trainUval finetune each added smaller but reliable gains that compounded.

What *did not* work was as informative as what did. CutMix hurt fine-grained classification — mixing patches between birds destroys the exact local cues the model needs to discriminate species. The PSM attention-rollout variant added implementation complexity without measurable benefit; the global-pooled feature was already attending to the right region once the crop-refine was in place. EfficientFormerV2-S1 never beat TinyViT-micro at a similar parameter budget, so we demoted it (and later EfficientNet-B0) to a localization-only role inside the ensemble.

5 What We Would Do Differently

Use a dedicated detection head from the start. Regressing a single box from a global-pooled feature is crude — there is no spatial resolution in the prediction. A 4×4 or 8×8 spatial feature map with anchor-free decoding (FCOS-style) would localize better at minimal parameter cost, and since crop-refine drove most of our gains, better boxes would compound directly into better classification.

Decouple classification and localization earlier. We jointly optimized both heads with a fixed $2\times$ bbox weight, but the tasks can conflict (classification wants pose-invariant features, localization wants position-sensitive ones). Training a dedicated localization model first, or running a curriculum (bbox-only for the first few epochs, then joint), might improve both heads simultaneously.

Spend more time tuning augmentation. We only used hflip, color jitter, and bbox-safe crop. Mosaic, box-preserving RandAugment, and feature-level mixup could push classification accuracy further at zero parameter cost, which is essentially free under our hard cap.

Run a systematic hyperparameter search. The final config (`img_size` 256, 12 epochs, GIoU, EMA, label smoothing 0.05, bbox weight 2.0, inference temperature 0.9) was found by iterative trial-and-error. A small grid over `bbox_weight` $\in \{1, 2, 5\}$ and inference temperature $\in \{0.7, 0.9, 1.0\}$ would likely yield another $\sim 0.5\%$ essentially for free.

Explore TTA more aggressively. We used only hflip plus crop-refine. Multi-scale TTA (224 and 288) plus center/corner crops are standard, cheap, and typically worth another 0.5–1% with no retraining.